Acceptable Use Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC1.1 (Integrity + ethical values) · CC1.4 (Attracts + develops competent people) · ISO 27001 A.5.10 (Acceptable use of information) **Companion documents:** Access Control Policy · Password + Credential Policy · Confidentiality Policy · `feedback_railcall_culture.md` (the internal cultural memory this policy codifies).

1. Purpose

State the standard of conduct expected of every person with access to RailCall systems, customer data, or third-party accounts held in RailCall's name. Deliberately short — a two-person team doesn't need a novel, but the standard still has to be written down for it to be enforceable.

2. Scope

Every operator with any credential in the systems listed in Access Control Policy §6 or Vendor Management Policy §3. Currently: Sami and Pat. New operators sign this policy (electronic acknowledgement) as part of access provisioning.

3. The core rules

3.1 No fake-green

No commit, no policy, no customer-facing claim shall assert something we cannot demonstrate on the record. Every "IMPLEMENTED" cites the code. Every "in progress" names an owner + a date. Every gap is a gap, not a euphemism.

This is the load-bearing rule. Every other rule follows from it.

Applies to: commit messages, docs, marketing pages, investor materials, compliance policies, incident notes, customer emails.

3.2 Evidence before prose

Read the code before you write the finding. Read the log before you write the post-mortem. If the finding can't cite a source, the finding is wrong.

Rewriting a finding to match the code after the fact is fine and expected. Writing a finding first and then bending the description to protect it is falsification.

3.3 Secrets never in text

No API key, seed hex, password, or token ends up in email, Slack, chat, git, issue tracker, or any surface an attacker with search access can trawl. Signal disappearing messages + 1Password shared vault + `~/*key*.txt` at 0600 are the accepted transports.

If a secret DOES land in a durable channel by accident: rotate it within 4 hours (per Incident Response Policy §3 SEV-2 response window), log the incident, do the post-mortem.

Exception: dev/test keys explicitly scoped to a throwaway resource, when the resource is documented as throwaway. Even then: rotate on any doubt.

3.4 Dry-run by default

Any command touching production state defaults to dry-run when the flag exists. Only strip the dry-run when the plan has been read.

Applies to: station releases (leak-gate + fresh-install smoke run before pin change), `install.sh` re-pins (byte-compare before commit), Render env-var changes (single-variable edits, never bulk paste), Stripe operations (test first when the API supports it), any bash `rm -rf` (list before delete).

3.5 One SoT per fact

Every fact lives in one authoritative place. When a fact changes there, every downstream copy updates in the same change. Downstream copies drift; SoT never does.

Concrete SoTs: - **Source code:** `github.com/patl4588/railcall-core|engine|contrib`. Not local checkouts, not release tarballs. - **Compliance artifacts:** `railcall-core/compliance/` in git. Probo is a viewer, not the SoT. - **Customer entitlements:** Render Postgres. Local install caches are derived state. - **Issuer signing seed:** Sami's 1Password + paper backup. Every other copy is either the ceremony's ephemeral output (shred after use) or an env-var copy set from the SoT. - **This policy set:** git. See `compliance/policies/README.md`.

3.6 Least privilege by default

Every new credential is scoped as narrowly as the task allows. Restricted keys preferred over standard keys. Read-only preferred over write. When "just this once" pressure argues for a broader scope, log the decision in the change record so future review can audit it.

3.7 No unilateral destruction of shared state

`git push --force` to `main`, `git reset --hard` on a shared branch, dropping a production table, deleting a Stripe customer, revoking another operator's access — none happen without either (a)

written acknowledgement from another operator, or (b) an active incident where the operator is the IC and logs the action in the incident note.

Local destruction (your own branch, your own checkout, your own machine) is your call. Shared destruction is a two-person decision.

3.8 Honest communication with customers + investors + regulators

Same rule as §3.1 externally. If a customer asks whether we're SOC 2 compliant and we're not: we say "not yet, targeting Q4 2026, here's the readiness posture." Not "we're in progress" (which reads as "will be soon"). Not "SOC 2 controls implemented" (which reads as "the audit is done").

---

4. Prohibited uses

Rare that these need saying but stating them makes enforcement cleaner:

- **Personal use of customer data.** Zero. Not for demo videos, not for screenshots, not for tests.
- **Personal use of RailCall infrastructure.** Rendering personal websites on the VPS, running personal databases on Render, using the Stripe account for personal invoices. All out.
- **Using operator credentials for anything outside their scope.** Sami's GitHub PAT is for RailCall repos. Pat's Render access is for RailCall services. Not for side projects.
- **Sharing credentials.** Ever. Not even with each other. Access is granted per-person via the account's own team/collab mechanism.
- **Bypassing controls to "just get it done."** Skipping tests to ship faster. Disabling the leak gate to publish a tarball. Answering a compliance question with what the customer wants to hear instead of what's true. All violate this policy AND every other policy in the set.

---

5. Escalation

If you see any of the following, escalate immediately (Signal call to the other operator + note in `~/incidents/`):

- Another operator violating §3 or §4.
- Yourself under time pressure to violate §3 or §4.
- A tool or automation about to violate §3 or §4 (an AI agent making a bad commit, a script about to `git push --force`).

Escalation is not blame; escalation is the safety net. The cost of a false escalation is a Signal call. The cost of a missed one is a preventable incident.

6. Enforcement

- **First violation:** discussed, documented, corrective action taken. No formal sanction — the person meant well, the process failed.
- **Repeated violation of the same rule:** access to the affected system scoped down + a written plan for how the process changes to prevent recurrence.
- **Deliberate violation with knowledge:** access revoked, considered grounds for removal from the team.

Team of two operating under trust: the enforcement mechanism here is the relationship, not a formal HR process. When the team grows, this section tightens.

7. Acknowledgement

Every operator signs (electronic acknowledgement in Probo, or paper) at onboarding and re-signs at each major revision of this policy. Sami's adoption of this v1 is recorded in Probo document `Access Control Policy` §8 and mirrored to `compliance/policies/README.md`.

8. Related documents

- Access Control Policy §6 — the systems this policy governs conduct within.
- Password + Credential Policy — the per-credential hygiene that §3.3 relies on.
- Confidentiality Policy — the customer-data-specific extension of §4.
- Change Management Policy §6 — the standing merge authorization + emergency change path that §3.6 (least privilege) interacts with.
- `feedback_railcall_culture.md` (Claude auto-memory) — the running record of cultural rules this policy codifies.